

Primitive Ring Placement Index Specification

Stage 5-P0/P1: Stable-Key Lookup and Exact Periodic Ring Placement

mdstats

2026-07-18

Contents

1	Purpose and gate status	2
2	Motive	2
3	Algorithmic provenance and attribution	2
4	Dependency boundary	3
5	Mathematical conventions	3
5.1	Periodic quotient edge	3
5.2	Canonical ring representative	3
5.3	Edge anchor of one canonical ring step	4
5.4	Recovering a translated ring placement	4
6	API standard	4
6.1	Exceptions	4
6.2	LiftedEdgeInstanceRef	4
6.3	RingPlacement	5
6.4	RingEdgePlacement	5
6.5	PrimitiveRingIndex	5
6.6	build_primitive_ring_index	5
6.7	ring_placements_covering_edge	6
7	S5-P0 structural hardening	6
8	Determinism and identity	6
9	Completeness semantics	7
10	Edge cases	7
10.1	Reverse traversal	7
10.2	Boundary-crossing rings	7
10.3	Parallel edges	7
10.4	Two-member rings	7
10.5	Repeated quotient-edge orbit	7
10.6	Self-image edge	7
10.7	Bound refinement	7
10.8	Source mismatch	8
11	Test gate	8
11.1	Unit tests	8
11.2	Na-LTA integration gate	8

1 Purpose and gate status

This document specifies the first implementation gate after the completed Stage-4R primitive-ring catalog:

`mdstats/analysis/primitive_ring_index.py`

The gate covers only:

- **S5-P0**: minimal hardening required by downstream source-bound ring lookup;
- **S5-P1**: exact translated primitive-ring placement around a requested lifted framework-edge instance.

The package version for this runtime/API revision is:

`mdstats 0.19.13a0`

This stage deliberately does **not** implement periodic-net symmetry, ring strength, geometric face construction, natural tiling, or a general periodic chain algebra. Those stages remain downstream.

Revision 0.19.13a0 API note. Stage-5 cleanup source-binds cross-module identities by topology digest, replaces bare dense edge indices in `LiftedEdgeInstanceRef` with stable `FrameworkEdgeKey`, moves `RingPlacement` to `periodic_cycle.py`, hides occurrence buckets as implementation detail, and adds ordered canonical/translated support accessors. The authoritative shared contract is `stage5_periodic_infrastructure_spec.md`.

2 Motive

`PrimitiveRingCatalog` stores one canonical representative for each translation orbit of a bounded zero-winding primitive ring. Downstream algorithms need to refer to physical translated copies of those rings without creating a second scientific ring catalog.

The immediate required query is:

Given one exact lifted physical instance of a framework edge, which translated primitive-ring placements represented by the source catalog contain that edge?

This query is required by later work on:

- exact automorphism-induced ring actions;
- translated smaller-ring placement during strong-ring decomposition;
- occurrence-level ring incidence;
- later embedded-face and cell-complex construction.

The design must preserve stable structural identity across changes in the dense local `ring_id` assignment. Persistent identity is therefore based on `PrimitiveRingKey`, while dense IDs remain source-catalog-local acceleration handles only.

3 Algorithmic provenance and attribution

The periodic representation follows the quotient-graph / edge-vector viewpoint used for periodic nets by Chung, Hahn, and Klee [1] and further discussed by Klee [2]. A quotient edge carries an integer lattice translation; one physical edge instance is obtained by adding a common lattice translation to that edge.

The specific Stage-5 placement index, stable-key lookup, canonical edge-anchor convention, occurrence records, and deterministic query API are `mdstats`-specific adaptations. This module does not transcribe an algorithm from either reference.

Implementation comments adjacent to the physical-edge anchor and translation recovery formulas must retain the attribution to [1,2].

4 Dependency boundary

The dependency direction is:

```
FrameworkTopology
  |
  v
PrimitiveRingCatalog
  |
  v
PrimitiveRingIndex  (transient, source-bound)
  |
  +--> exact translated ring placements
  +--> future symmetry occurrence actions
  +--> future strong-ring support search
```

`PrimitiveRingIndex` is an acceleration and identity layer over one existing `PrimitiveRingCatalog`. It is not a second ring catalog.

5 Mathematical conventions

5.1 Periodic quotient edge

Let a canonical quotient edge be

$$e = (i, j, \Delta), \quad \Delta \in \mathbb{Z}^3,$$

where the canonical physical instance anchored at image $\mathbf{a}$ joins

$$(i, \mathbf{a}) \longrightarrow (j, \mathbf{a} + \Delta).$$

The orientation-independent physical edge-instance identity is

$$(e, \mathbf{a}).$$

The anchor $\mathbf{a}$ always refers to the image of canonical endpoint i , regardless of traversal direction.

5.2 Canonical ring representative

Every `PrimitiveRing` already stores one deterministic canonical lifted representative:

$$\widehat{R}(q) = (\widehat{V}(q), \widehat{E}(q)),$$

where q is its `PrimitiveRingKey` and the first lifted vertex is anchored at image

$$\mathbf{0} = (0, 0, 0).$$

Stage 5 must reuse this stored representative. It must not independently recanonicalize the ring.

A translated placement is

$$R(q, \mathbf{t}) = \widehat{R}(q) + \mathbf{t}, \quad \mathbf{t} \in \mathbb{Z}^3.$$

5.3 Edge anchor of one canonical ring step

For canonical ring step k , let its source lifted vertex be

$$(v_k, \mathbf{s}_k),$$

and let its framework edge have canonical shift Δ_k .

If the ring traverses the canonical edge forward,

$$a_k = \mathbf{s}_k.$$

If it traverses the canonical edge in reverse,

$$a_k = \mathbf{s}_k - \Delta_k.$$

Thus a_k is orientation-independent physical-edge identity in the canonical ring representative.

5.4 Recovering a translated ring placement

Suppose the requested physical edge instance is anchored at $\mathbf{a}$ and a canonical ring occurrence of the same edge orbit is anchored at a_k . The unique translation aligning that occurrence to the request is

$$\mathbf{t} = \mathbf{a} - a_k.$$

Translation preserves the ring step orientation and all relative image shifts. No shortest-path search or graph traversal is required.

6 API standard

The API introduced by this gate is intentionally small and provisional. It may be promoted or refactored only after the Stage-5 symmetry prototype exercises it.

6.1 Exceptions

```
class PrimitiveRingIndexError(ValueError):  
    """Base exception for primitive-ring index and placement operations."""  
  
class PrimitiveRingIndexInputError(PrimitiveRingIndexError):  
    """Raised when a source catalog, key, edge instance, or index is invalid."""
```

6.2 LiftedEdgeInstanceRef

```
@dataclass(frozen=True, order=True, slots=True)  
class LiftedEdgeInstanceRef:  
    topology_graph_digest: str  
    edge_key: FrameworkEdgeKey  
    anchor_shift: tuple[int, int, int]
```

Meaning:

- `topology_graph_digest` source-binds the record;
- `edge_key` is the complete stable decorated framework-edge identity;
- `anchor_shift` is the image of canonical endpoint `vertex_i`.

Bare dense edge indices are intentionally excluded from cross-module physical edge identity.

6.3 RingPlacement

Defined in `periodic_cycle.py`:

```
@dataclass(frozen=True, order=True, slots=True)
class RingPlacement:
    topology_graph_digest: str
    ring_key: PrimitiveRingKey
    image_shift: tuple[int, int, int]
```

The digest and stable ring key jointly source-bind the placement. A placement from another topology must be rejected before lookup. Parametrization is handled separately by `CycleParameterization`.

6.4 RingEdgePlacement

```
@dataclass(frozen=True, order=True, slots=True)
class RingEdgePlacement:
    placement: RingPlacement
    step_index: int
    orientation: Literal[-1, 1]
```

This record means that `placement` contains the requested physical edge instance at `step_index`, traversed with the recorded source-ring orientation.

6.5 PrimitiveRingIndex

```
@dataclass(frozen=True, slots=True)
class PrimitiveRingIndex:
    catalog: PrimitiveRingCatalog
    ring_keys: tuple[PrimitiveRingKey, ...]

    @property
    def edge_count(self) -> int: ...

    @property
    def occurrence_count(self) -> int: ...

    def ring_id_for_key(self, key: PrimitiveRingKey) -> int: ...
    def ring_for_key(self, key: PrimitiveRingKey) -> PrimitiveRing: ...
    def edge_index_for_key(self, key: FrameworkEdgeKey) -> int: ...
    def edge_key_for_index(self, edge_index: int) -> FrameworkEdgeKey: ...
    def canonical_edge_instance(self, key, step_index) -> LiftedEdgeInstanceRef: ...
    def canonical_edge_instances(self, key) -> tuple[LiftedEdgeInstanceRef, ...]: ...
    def translated_edge_instances(self, placement) -> tuple[LiftedEdgeInstanceRef, ...]: ...
```

The occurrence buckets and dense edge-index tables are private acceleration details. `canonical_edge_instances()` is the supported ordered physical-support accessor.

6.6 build_primitive_ring_index

```
def build_primitive_ring_index(
    catalog: PrimitiveRingCatalog,
) -> PrimitiveRingIndex:
    ...
```

Input:

- one validated `PrimitiveRingCatalog`.

Output:

- one transient deterministic `PrimitiveRingIndex`.

Complexity:

$$O\left(\sum_{R \in \mathcal{R}} |R|\right)$$

time and storage, excluding the already-owned source catalog.

6.7 ring_placements_covering_edge

```
def ring_placements_covering_edge(
    index: PrimitiveRingIndex,
    edge_instance: LiftedEdgeInstanceRef,
) -> tuple[RingEdgePlacement, ...]:
    ...
```

Input constraints:

- `index` must be a valid `PrimitiveRingIndex`;
- `edge_instance.topology_graph_digest` must equal the owning index digest;
- `edge_instance.edge_key` must exist in the source framework graph;
- `anchor_shift` uses the source catalog's exact lattice basis.

Output:

- every represented translated ring occurrence whose indicated canonical step coincides with the requested physical edge instance;
- deterministic sorted tuple;
- no completeness claim beyond the source catalog's ring family and bounded completeness status.

For occurrence anchor a_k and requested anchor a , the returned placement uses

$$\mathbf{t} = a - a_k.$$

7 S5-P0 structural hardening

The existing `PrimitiveRingCatalog` validation must additionally verify that `ring.vertex_walk` and `ring.steps` describe one continuous lifted cycle against the catalog's stored edge keys.

For each step:

- forward traversal must start at `vertex_i` and end at `vertex_j` with image increment `image_shift`;
- reverse traversal must start at `vertex_j` and end at `vertex_i` with image decrement `image_shift`;
- the final step must close onto the first lifted vertex.

This validation changes no serialization schema and no scientific result. It only rejects internally inconsistent catalog records earlier.

8 Determinism and identity

The following are normative:

1. `PrimitiveRingKey`, not `ring_id`, is the stable ring-orbit reference.
2. Dense IDs are valid only inside their owning catalog.
3. Index construction never mutates or renumbers source rings.
4. Ring placements use integer lattice translations only.
5. Edge-instance anchors are orientation-independent.

6. Parallel framework edges remain distinct because complete `FrameworkEdgeKey` identity and `FrameworkEdgeKey` identity remain distinct.
7. Repeated occurrences of one quotient edge orbit in a ring are not collapsed.
8. Query results are sorted deterministically.

9 Completeness semantics

The index is exact **relative to its source catalog**.

If the source catalog is complete for all primitive/no-shortcut rings through size K , then the index query is complete for translated placements of those cataloged ring orbits.

If the source catalog is truncated or represents only the legacy edge-shortest subset, the placement query remains exact for represented rings but must not upgrade the source family or completeness claim.

Formally, for source ring set $\mathcal{R}_C$ and requested physical edge instance e_a , the query returns exactly

$$\{(R, \mathbf{t}, k) : R \in \mathcal{R}_C, \text{ step } k \text{ of } R + \mathbf{t} \text{ is } e_a\}.$$

10 Edge cases

10.1 Reverse traversal

Reverse traversal must subtract the edge translation before forming the orientation-independent anchor. Using the source lifted image directly for both orientations is incorrect.

10.2 Boundary-crossing rings

Canonical ring vertices may carry nonzero images even though total winding is zero. Placement translation must shift every lifted image uniformly; wrapped coordinates must not be used.

10.3 Parallel edges

Two projected edges with the same endpoint atoms but different linker path, translation, or rule identity are distinct edge orbits. They must never be merged by endpoint pair alone.

10.4 Two-member rings

Two-member rings formed by parallel edges are valid Stage-4R objects. The index must retain the two distinct step occurrences and exact edge identities.

10.5 Repeated quotient-edge orbit

A lifted-simple ring may in principle traverse translated instances of the same quotient edge orbit more than once. `edge_to_occurrences` therefore indexes step occurrences, not merely ring IDs.

10.6 Self-image edge

One-member rings remain unsupported by Stage 4. A nonzero self-image quotient edge may still appear in other supported cycles; its physical instance anchor must follow the same canonical-endpoint convention.

10.7 Bound refinement

Increasing the primitive-ring bound may change dense `ring_id` assignments. Downstream persistent references must therefore retain `PrimitiveRingKey` and source digest, then resolve the current dense ID through the index.

10.8 Source mismatch

`LiftedEdgeInstanceRef` contains a dense edge index and is intentionally source-bound. Passing an instance created for another topology/catalog is a caller error unless an explicit periodic-net mapping has first transformed it.

11 Test gate

The focused gate must pass before Stage S5-P2 begins.

11.1 Unit tests

1. stable-key lookup resolves every source ring;
2. unknown keys are rejected;
3. canonical edge-instance anchors agree with stored lifted walks;
4. forward and reverse traversal anchors are both correct;
5. a translated target edge recovers the same common ring translation;
6. boundary-crossing zero-winding rings translate correctly;
7. parallel edges remain distinct in a two-member ring;
8. one physical edge shared by several rings returns every represented placement;
9. invalid dense edge indices are rejected;
10. malformed step/vertex-walk continuity is rejected by catalog validation.

11.2 Na-LTA integration gate

Using the checked-in `na_lta_framework_topology.json` fixture:

$$36 \times 4R + 40 \times 6R + 6 \times 8R = 82 \text{ rings.}$$

The total canonical ring-step occurrence count must be

$$36 \cdot 4 + 40 \cdot 6 + 6 \cdot 8 = 432.$$

For every one of the 432 canonical occurrences:

1. query its own canonical physical edge instance;
2. verify that the result contains the same `PrimitiveRingKey`;
3. verify the same `step_index`;
4. verify zero placement translation.

A nonzero common translation applied to sampled canonical edge instances must recover the identical translation in the returned `RingPlacement`.

12 Gate acceptance

S5-P0/P1 passes only if:

- the specification exists in Markdown and PDF;
- all new focused tests pass;
- the existing primitive-ring focused tests pass;
- the Na-LTA 82-ring / 432-occurrence integration gate passes;
- no Stage-4R ring counts, keys, digests, or serialization outputs change;
- no broad periodic helper refactor is introduced yet.

After this gate, implementation proceeds to S5-P2: exact automorphism-induced mapping of ordered ring occurrences using known validated periodic transformations.

13 References

1. S. J. Chung, Th. Hahn, and W. E. Klee, "Nomenclature and generation of three-periodic nets: the vector method," *Acta Crystallographica Section A* **40**, 42-50 (1984). DOI: [10.1107/S0108767384000088](https://doi.org/10.1107/S0108767384000088).
2. W. E. Klee, "Crystallographic nets and their quotient graphs," *Crystal Research and Technology* **39**, 959-968 (2004). DOI: [10.1002/crat.200410281](https://doi.org/10.1002/crat.200410281).